

E-Commerce Payment Gateway - Integration Architecture & A

Confidential - Internal Use Only

Version 1.0

1. System Context

The ECO Payment Gateway sits between Merchant Applications (frontend/backend) and Financial Networks (Visa, Mastercard, Amex, PayPal).

External Systems: Merchant Storefront (sends payment requests), Acquiring Banks (processes authorizations), Card Networks (routes transactions), XE Currency API (exchange rates), Onfido (KYC verification), Cardinal Commerce (3DS2).

Internal Services: Payment Processor (orchestration), Fraud Engine (risk scoring), Tokenization Service (PCI vault), Settlement Engine (daily settlement), Notification Service (webhooks/email), Merchant Portal (dashboard).

2. API Gateway & Authentication

REQ-ECO-070: All merchant API calls must authenticate via Bearer token (API key) in the Authorization header.

REQ-ECO-071: API versioning uses URL path prefix (/v1/, /v2/) with 12-month deprecation notice for old versions.

REQ-ECO-072: Rate limiting enforced at API gateway: 100 requests/second per merchant in production, 20 req/sec in sandbox.

REQ-ECO-073: All API responses follow a consistent envelope: {status, data, error, request_id, timestamp}.

3. Payment Processor <-> Card Network Integration

REQ-ECO-074: Authorization requests to Visa/MC use ISO 8583 message format via the acquiring bank's payment gateway.

REQ-ECO-075: Connection to acquiring bank uses dedicated VPN tunnel with mutual TLS authentication.

REQ-ECO-076: Timeout for authorization response is 5 seconds. On timeout, transaction marked as 'Unknown' and queried via reversal/inquiry.

REQ-ECO-077: End-of-day batch settlement file sent to acquiring bank in ISO 8583 format at 00:00 UTC.

4. Fraud Engine <-> ML Model Serving

REQ-ECO-078: Fraud scoring model served via AWS SageMaker real-time endpoint with 100ms SLA.

REQ-ECO-079: Model features extracted from Kafka stream: transaction velocity (last 1hr, 24hr), geolocation delta, device fingerprint match, amount deviation from customer average.

REQ-ECO-080: Model retraining pipeline runs monthly using Amazon SageMaker Pipelines with labeled fraud/legitimate transaction data from S3.

REQ-ECO-081: A/B testing framework allows running challenger models on 10% of traffic before promotion.

5. Webhook Delivery Architecture

REQ-ECO-082: Webhook events published to Kafka topic 'merchant-events'. Consumer service reads events and dispatches HTTP POST to merchant endpoints.

REQ-ECO-083: Each webhook delivery attempt is logged with: event_id, merchant_id, endpoint_url, http_status, response_time, attempt_number.

REQ-ECO-084: Dead letter queue (DLQ) captures events that fail all 5 retry attempts. DLQ items

visible in merchant dashboard for manual replay.

6. Sequence Flow: Standard Payment

1. Merchant Backend -> ECO API: POST /v1/payment-intents {amount, currency, merchant_id}
2. ECO API -> Payment Processor: Create PaymentIntent record in DB, return {client_secret}
3. Merchant Frontend -> Hosted Checkout: Initialize iframe with client_secret
4. Hosted Checkout -> Tokenization Service: POST /internal/tokenize {encrypted_card_data}
5. Tokenization Service -> Hosted Checkout: {payment_token}
6. Hosted Checkout -> Payment Processor: POST /internal/process {payment_token, client_secret}
7. Payment Processor -> Fraud Engine: POST /internal/score {transaction_features}
8. Fraud Engine -> Payment Processor: {risk_score: 32, decision: 'approve'}
9. Payment Processor -> 3DS Service: POST /internal/authenticate (if card enrolled)
10. 3DS Service -> Payment Processor: {eci_indicator, authentication_status}
11. Payment Processor -> Acquiring Bank: ISO 8583 Authorization Request
12. Acquiring Bank -> Payment Processor: Authorization Response {auth_code, response_code}
13. Payment Processor -> Notification Service: Publish 'payment.success' event to Kafka
14. Notification Service -> Merchant Webhook: POST https://merchant.com/webhook {event_type, payment_intent}